# Memory Court: Tamper-Evident Evidence Chains and Human Adjudication for AI Agent Memory

Anonymous

**Abstract**

AI agents accumulate persistent memory across sessions, but no existing system provides verifiable evidence that a stored memory was not tampered with, or a replayable record of how conflicting memories were adjudicated. We present Memory Court, a governance layer for agent memory that provides three capabilities: (1) a SHA-256 hash-chained audit log that detects any post-hoc modification of memory records (tamper-evident by construction), (2) a case-file command that reconstructs the full provenance of any memory—its write history, supersession chain, conflict adjudications, and Q-value learning trace—as a human-readable dossier, and (3) a browser-verifiable evidence export (`evidence.zip`) that enables third-party auditors to verify the hash chain without installing any software, directly addressing EU AI Act Article 12 record-keeping requirements. We evaluate Memory Court on a production deployment (1,864 active memories, 13 agent sources, 26 days) and demonstrate: 14 integration tests covering 6 dimensions (hash chain verification, tamper detection, write-time conflict, evidence export, supersession adjudication, and end-to-end integration), a case study of cross-agent contradiction resolution (packy entity: 60% contradiction rate resolved to 0% via supersession adjudication), and a controlled tamper-detection experiment over 2,346 audit entries where the hash chain correctly detects all 5 of 5 anchored-row modification scenarios. We release the system as Apache-2.0 open source with an interactive Playground demonstrating the full court workflow in a browser.

## 1 Introduction

AI agents that operate over extended periods accumulate memory: user preferences, system configurations, past decisions, and learned procedures. As these agents proliferate in regulated domains—healthcare, finance, legal—the ability to *verify* what the agent "knew" and "when" becomes a compliance requirement, not a nice-to-have. The EU AI Act (Article 12) mandates that high-risk AI systems maintain automatically generated event logs sufficient to reconstruct the traceability of the system's functioning.

Current agent memory systems—mem0 [Chhikara et al.(2025)], Zep [Rasmussen et al.(2025)], Letta [Packer et al.(2023)]—do not address this requirement. They store memories but provide no cryptographic guarantee that stored records have not been modified after the fact. They resolve conflicts between memories but provide no replayable record of *who* decided, *when*, and *based on what evidence*. They support retrieval but provide no exportable dossier that a third-party auditor could verify independently.

We present Memory Court, a governance layer that upgrades agent memory from "a database of facts" to "a court of record." Memory Court provides:

1. **Hash-chained evidence log**: Every memory operation (write, correct, supersede, retire, search) is logged in an append-only audit table. Every 10 entries, a SHA-256 anchor is computed over the chain. Any post-hoc modification of a logged entry breaks the chain and is detected by verification.

2. **Case-file reconstruction**: Given any memory UID, `memtether court <uid>` reconstructs the complete judicial dossier: current content with confidence and Q-value, bi-temporal validity evidence (T-axis and T′-axis), full provenance chain (who wrote, who corrected, who searched), conflict adjudication records, and integrity proof.

3. **Browser-verifiable evidence export**: `memtether court <uid> --export evidence.zip` produces a ZIP containing the machine-readable case file (`case.json`), the hash-chained audit trail (`chain.jsonl`),

and a self-contained HTML page (`verify.html`) that recomputes the hash chain in the browser via `crypto.subtle`—no software installation required.

# 2 Background and Motivation

## 2.1 Agent Memory Systems

mem0 [Chhikara et al.(2025)] provides hosted memory with LLM-based fact extraction; when `update()` is called, the old embedding is replaced in-place with no audit trail. Zep [Rasmussen et al.(2025)] offers a temporal knowledge graph but no cryptographic integrity guarantee. Letta [Packer et al.(2023)] manages hierarchical memory with no evidence export. LangMem [Liu et al.(2025)] integrates memory into LangGraph without tamper detection.

Caura (formerly MemClaw) [Margalit et al.(2026)] is the most directly related system: it provides governed shared memory with contradiction detection and supersession, deployed in production at eToro. However, Caura's audit log is a standard database log without cryptographic chaining, and it provides no evidence export mechanism.

## 2.2 Regulatory Context

The EU AI Act (Regulation 2024/1689) establishes mandatory record-keeping requirements. Article 12 requires that high-risk AI systems "technically allow for automatic logging of events" over the lifetime of the system. For GPAI models, compliance is required by 2027-08-02. Memory Court's evidence export directly implements this requirement at the memory layer: the `evidence.zip` package contains the hash-chained audit trail sufficient to "ensure the traceability of the system's functioning" as required by Art. 12(1).

## 2.3 Competitive Gap

We conducted a systematic search of GitHub repositories and arXiv papers (October 2026) for systems combining memory-level hash chaining, exportable evidence packages, and human adjudication UI. The results are shown in Table 1.

Table 1: Competitive gap analysis (October 2026).

| Capability | Tencent (27.7K) | memtrace (486) | icarus (290) | halo-record (83) | **MemTether** |
|---|---|---|---|---|---|
| Supersession chains | – | – | ✓ | – | ✓ |
| Bi-temporal model | – | ✓ | – | – | ✓ |
| Conflict adjudication | – | – | – | – | ✓ |
| Memory-level provenance | – | – | ✓ | – | ✓ |
| **Hash-chained audit** | – | – | – | runtime | ✓ |
| **Evidence export** | – | – | – | runtime | ✓ |
| **Adjudication UI** | – | – | – | – | ✓ |

The combination of all three capabilities—memory-level hash chaining, exportable evidence, and adjudication UI—is provided by no existing system. halo-record implements hash chaining at the runtime action layer but does not touch memory content.

Recent tools have partially addressed adjacent problems: TradeMemory Protocol [Trademem(2026)] implements SHA-256 chains for trading agents but is domain-specific. ZizkaDB [ZizkaDB(2026)] provides tamper-evident audit trails but focuses on decision logs rather than memory content. PACMI [PACMI(2026)] performs provenance-aware memory invalidation without cryptographic chaining. TRACE [TRACE(2026)] governs memory validity in multi-agent systems without hash chains. Memora [Memora(2026)] provides supersession lineage without evidence export or adjudication UI. The combination of all three capabilities—memory-level hash chaining, browser-verifiable evidence export, and human adjudication UI—is provided by no existing system.

# 3 System Design

## 3.1 Architecture Overview

Memory Court is implemented as a governance layer atop MemTether's existing memory store. It reuses four existing components without modification: the `audit_log` table (every operation is already logged), the `supersessions` table (adjudication records), the `conflict_reviews` table (human decisions), and the bi-temporal model (T-axis for fact validity, T′-axis for system awareness). The only new component is the hash chain.

## 3.2 Hash-Chained Evidence Log

Every memory operation is recorded in the `audit_log` table with fields `(id, op, target, agent, detail, ts)`. Every 10 entries, a SHA-256 anchor is computed:

$$\text{anchor}_b = \text{SHA256}\big(\text{anchor}_{b-1} \,\|\, H(e_{10b-9}) \,\|\, H(e_{10b-8}) \,\|\, \cdots \,\|\, H(e_{10b})\big) \tag{1}$$

where $H(e_i) = \text{SHA256}(\text{id}_i \| \text{op}_i \| \text{target}_i \| \text{agent}_i \| \text{detail}_i \| \text{ts}_i)$ is the per-entry hash and $\|$ denotes concatenation. Every 10 entries form one batch; the batch hash is chained to the previous anchor, so any modification of any logged entry changes the covering anchor. Anchors are stored in an `audit_anchors` table. Verification recomputes the entire chain from the genesis anchor and reports ✓ or × per entry.

**Finding 1** (Tamper-Evident by Construction)**.** *Any modification of a logged entry (content, timestamp, agent, or deletion) changes the SHA-256 hash of all subsequent entries, causing chain verification to fail from the modified entry onward. This is a property of the construction, not of the implementation.*

## 3.3 Case-File Reconstruction

Given a memory UID, the case file reconstructs five sections:

1. **Current content**: The active version with confidence score and Q-value (learning trace from use).
2. **Temporal evidence**: The bi-temporal validity period—when the fact was true in the world (T-axis) versus when the system learned it (T′-axis).
3. **Provenance chain**: The complete operation history from write through corrections to current state, with agent identity and timestamp for each step.
4. **Conflict adjudication**: Any supersession decisions involving this memory, including the reason and the adjudicating agent.
5. **Integrity proof**: The hash-chain anchors covering this memory's audit entries, with verification status.

## 3.4 Evidence Export

The `--export` flag produces a ZIP package containing:

- `case.json`: Machine-readable case file (all five sections above).
- `chain.jsonl`: All relevant audit log entries and hash anchors.
- `verify.html`: A self-contained HTML page that recomputes the SHA-256 chain in the browser using `crypto.subtle`, displaying ✓ or × per entry. No server, no installation, no trust in the exporter required—the auditor verifies the chain independently.

### 3.5 Interactive Playground

A browser-based Playground (`https://memtether.github.io/MemTether/court/`) demonstrates the full workflow: write a memory, correct it, trigger a conflict, adjudicate via supersession, verify the hash chain, and export the evidence package. The Playground uses sql.js (SQLite compiled to WASM) and `crypto.subtle` (browser-native SHA-256), requiring zero backend infrastructure.

## 4 Evaluation

### 4.1 Production Deployment

Memory Court has been deployed in a production multi-agent environment for 26 days:

- 1,864 active memories from 13 distinct agent sources
- 2,346 audit log entries with 235 hash anchors
- 348 supersession events (adjudication decisions)
- 226 retired memories (explicit lifecycle management)

### 4.2 Tamper-Detection Experiment

We conducted a controlled tamper-detection experiment over 2,346 audit entries. We simulated 5 post-anchor modification scenarios (content change, timestamp change, agent identity change, entry deletion, and anchor tampering). The hash chain correctly detected all 5 of 5 anchored-row modifications. We note that this is an expected property of any correctly implemented hash chain; the contribution is the integration of this mechanism with self-contained evidence export in a production multi-agent deployment, not the detection mechanism itself.

### 4.3 Integration Testing

14 integration tests cover 6 dimensions of Memory Court's behavior:

Table 2: Test coverage by dimension.

| Dimension | Tests |
|---|---|
| Hash chain verification | 6 |
| Tamper detection | 2 |
| Write-time conflict | 1 |
| Evidence export | 2 |
| Supersession adjudication | 1 |
| End-to-end integration | 2 |
| Total | 14 |

All 14 tests pass. Five engineering-level gaps were identified (anchor tail coverage, concurrent anchor computation, database migration, 1000+ entry stress testing, and browser CI) but none affect the correctness of the core mechanism.

### 4.4 Case Study: Cross-Agent Contradiction Resolution

We present a case study from the production deployment. The *packy* entity (an API proxy service) accumulated 24 memories from 4 agent sources over 18 days. At the time of audit, 15 memories were active and retrievable: 2 asserted "available" while 3 asserted "banned" or "403", yielding a 60% contradiction rate. A downstream agent querying packy's status received contradictory evidence.

Memory Court resolved this through the supersession adjudication workflow: the most recent authoritative fact was identified via bi-temporal ordering, older conflicting facts were marked as superseded with explicit validity boundaries, and the contradiction rate dropped to 0%. The full adjudication record—who decided, when, based on what evidence, with hash-chain verification—is preserved in the audit log and exportable as an evidence package.

# 5 Discussion and Limitations

## 5.1 What Memory Court Does Not Do

Memory Court does not prevent memory poisoning attacks (an agent with write access can still write false memories); it makes such writes *detectable and attributable*. It does not automate conflict resolution; it provides the evidence infrastructure for *human* adjudication. It does not scale to distributed multi-machine deployments; it is designed for single-machine, multi-client scenarios.

## 5.2 Honest Limitations

- **Anchor tail coverage**: The last $< 10$ audit entries before the current position are not yet anchored. A crash in this window leaves entries temporarily unanchored (recovered by the next anchoring pass).
- **No Merkle tree**: Verification requires full-chain recomputation (O(n)); a Merkle tree would enable O(log n) single-entry proofs. This is a planned optimization, not a correctness issue.
- **Single-machine scope**: The hash chain is valid within one SQLite file. Cross-machine federation requires a distributed trust model not addressed here.
- **Single-author deployment**: All 13 agent sources are operated by one person, which limits the diversity of adversarial scenarios tested.
- **Small-sample case study**: The packy entity case study (60% contradiction rate) is based on a single entity with 15 active memories; the result is illustrative rather than statistically generalizable.
- **Free-tier LLM judge**: The downstream query in the case study used a free-tier API (GLM-5.3-flash); results may differ with paid or frontier-model judges.
- **Self-referential evaluation**: The 23 (now 14) integration tests and the tamper-detection experiment are self-designed and self-executed, with no external verification or artifact evaluation.
- **Zero external users**: The system has no real-world external users; all 13 agent sources are operated by the author.

# 6 Related Work

## 6.1 Tamper-Evident Logging

Certificate Transparency [Laurie et al.(2019)] pioneered publicly auditable append-only logs. MutMem [Chen et al.(2026)] applies SHA-256 commitments to memory mutation authorization. halo-record implements runtime-level hash chains for agent actions. Our contribution is applying these techniques at the *memory content* level with exportable evidence packages.

## 6.2 AI Act Compliance

Beyond Provenance [Beyond Provenance(2026)] develops an economic governance model. Lineage-Aware Memory Governance [Lineage(2026)] addresses enterprise memory through derivation-gated access control. Memory Court is the first system to implement Art. 12 record-keeping at the memory layer with browser-verifiable evidence.

### 6.3 Agent Memory Governance

Caura [Margalit et al.(2026)] provides governed shared memory without cryptographic integrity. AECG [AECG(2026)] defines memory pollution conceptually without a system implementation. Our companion system paper [Anonymous(2026)] presents the full MemTether architecture including the hash-chain implementation evaluated here.

## 7 Reproducibility

MemTether is Apache-2.0 open source at `https://github.com/MemTether/MemTether`. The Memory Court Playground is live at `https://memtether.github.io/MemTether/court/`. All tamper-detection and integration test results can be reproduced with:

```
# Hash chain verification (tamper detection)
git clone https://github.com/MemTether/MemTether.git
cd MemTether && pip install -e ".[server]"
python examples/audit_chain_demo.py
# Expected: verify PASS -> tamper -> verify FAIL

# Memory Court integration tests (14 tests, 6 dimensions)
pytest tests/test_memory_court.py tests/test_court_e2e_integration.py -v
# Expected: 14 passed

# Poisoning defense (8 vectors, including negative test_F)
pytest tests/test_poisoning_defense.py -v
# Expected: 8 passed

# Case file reconstruction
memtether court <fact-uid>
# Expected: 5-section dossier with hash chain status

# Evidence export (browser-verifiable)
memtether court <fact-uid> --export evidence.zip
# Open verify.html in browser -> chain verifies without installation
```

The production database is not publicly released; the Playground at the URL above uses a synthetic database with the same schema for interactive verification.

## 8 Conclusion

Memory Court upgrades agent memory from a passive data store to an active evidentiary system: every operation is hash-chained, every memory has a reconstructable case file, and every adjudication is exportable as a browser-verifiable evidence package. We identify this as a previously unaddressed gap in the agent memory landscape and release the complete implementation as Apache-2.0 open source.

## References


[Chhikara et al.(2025)] Chhikara, P. et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.

[Rasmussen et al.(2025)] Rasmussen, R. et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*, 2025.

[Packer et al.(2023)] Packer, C. et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.

[Liu et al.(2025)] Liu, S. et al. LangMem: Memory Management for LangGraph Agents. LangChain, 2025.

[Margalit et al.(2026)] Margalit, A. et al. Caura: Governed Shared Memory for Multi-Agent Fleets. *arXiv:2606.24535*, 2026.

[Laurie et al.(2019)] Laurie, B., Langley, A., Kasper, E. Certificate Transparency. RFC 6962, IETF, 2013.

[Chen et al.(2026)] Chen, S. et al. MutMem: Cryptographically Authorized Mutation for Persistent Agent Memory. *arXiv:2608.02843*, 2026.

[Beyond Provenance(2026)] Anonymous. Beyond Provenance: An Economic Governance Model for AI Memory. *arXiv:2609.25521*, 2026.

[Lineage(2026)] Anonymous. Lineage-Aware Memory Governance for Enterprise AI. *arXiv:2610.07258*, 2026.

[AECG(2026)] Anonymous. AECG: Asymmetric Experience Consolidation and Governance for Multi-Agent Memory. *arXiv:2610.05176*, 2026.

[Yadav(2026)] Yadav, N. Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors for AI Agents. *arXiv:2606.26511*, 2026.

[Trademem(2026)] Anonymous. TradeMemory Protocol: Tamper-Evident Audit Trail for AI Trading Agents. GitHub 1430 stars, 2026.

[ZizkaDB(2026)] Anonymous. ZizkaDB: Tamper-Evident Audit Trail for AI Agents. GitHub 130 stars, 2026.

[PACMI(2026)] Anonymous. PACMI: Provenance-Aware Cascading Memory Invalidation for Long-Term LLM Agents. *arXiv:2610.05732*, 2026.

[TRACE(2026)] Anonymous. TRACE: Governing Memory Validity in Evolving Multi-Agent Systems. *arXiv:2609.33517*, 2026.

[Memora(2026)] Anonymous. Memora: Persistent Collective Memory with Supersession Lineage. GitHub 739 stars, 2026.

[Anonymous(2026)] Anonymous. Local-First Governed Memory: Bi-Temporal Supersession, Hash-Chained Audit, and Evaluation-Layer Integrity for Multi-Client AI Agents. *arXiv*, 2026.